keyle

v0.3.0 2026-06-02 MIT

This package provides a simple way to style keyboard shortcuts in your documentation.

WESLEYEL
yxnian@outlook.com

Table of Contents

Part I

About

KEYLE is a library that allows you to create HTML `<kbd>` like keyboard shorts simple and easy.

The name, `keyle`, is a combination of `key` and `theme`.

This project is inspired by auth0/kbd[1] and dogezen/badgery[2]. Send them respect!

[1] http://github.com/auth0/kbd
[2] https://github.com/dogezen/badgery

Part II

Usage

II.1 Importing

KEYLE is imported using

```
#import "@preview/keyle:0.3.0"
```

II.2 Quick Start

Generate a keyboard renderer with `#config` function. Section Section III.1 lists available themes.

```
#let kbd = keyle.config()
#kbd("Ctrl","Shift","Alt","Del")

#let kbd = keyle.config(
  theme: keyle.themes.biolinum,
  delim: keyle.biolinum-key.delim_plus
)
#kbd("Ctrl", "Shift", "Alt", "Del")
```

Ctrl + Shift + Alt + Del

Ctrl + ⇧ + Alt + Del

There are `compact` and `delim` options to make the output more compact and change the delimiter between keys.

You can either use them in `#config` function or directly in generated `kbd` function.

```
#let kbd = keyle.config(compact: true, delim: "-")
#kbd("Ctrl", "Shift", "Alt", "Del")

#let kbd = keyle.config()
#kbd("Ctrl", "Shift", "Alt", "Del", compact: true, delim: "-")
```

Ctrl-Shift-Alt-Del

Ctrl-Shift-Alt-Del

Part III

Themes

III.1 Built-in Themes

Themes function are available at `#keyle.themes` dictionary.

`#keyle.themes.standard`	Home
`#keyle.themes.deep-blue`	Home
`#keyle.themes.type-writer`	Home
`#keyle.themes.minimal`	Home
`#keyle.themes.radix`	Home
`#keyle.themes.flowbite`	Home
`#keyle.themes.flowbite-dark`	Home
`#keyle.themes.daisy`	Home
`#keyle.themes.biolinum`	Home

III.2 Custom Themes

You can create your own theme by defining a function that takes a string and returns a themed content.

```
// https://www.radix-ui.com/themes/playground#kbd
#let radix_kdb(content) = box(
  rect(
    inset: (x: 0.5em),
    outset: (y: 0.05em),
    stroke: rgb("#1c2024") + 0.3pt,
    radius: 0.35em,
    fill: rgb("#fcfcfd"),
    text(fill: black, font: (
      "Helvetica Neue",
      "TeX Gyre Heros",
    ), content),
  ),
)
#let kbd = keyle.config(theme: radix_kdb)
#kbd("⌘ D") #kbd("^ F")
```

III.3 Extending a Theme with `.with()`

Every built-in theme is a `#keycap` or `#svg-keycap` factory with its style pre-bound. Because they are ordinary functions, you extend any of them using Typst's native `.with(..)`, without learning a bespoke override API. The text layer (`text-args`, `wrap`) and the cap layer (`fill`, `stroke`, `radius`, `raise`, …) stay separate.

```
#let rose = keyle.themes.flowbite.with(
  fill: rgb("#fee2e2"),
  stroke: rgb("#fca5a5"),
  text-args: (fill: rgb("#991b1b"), weight: "bold"),
)
#let kbd = keyle.config(theme: rose)
#kbd("Ctrl", "Shift", "K")
```

Part IV
Symbols

IV.1 Mac Keyboard Symbols

KEYLE provides symbols for Mac keyboard. You may need install `Fira Code` font to see the symbols correctly.

- Fira Code @ Google Fonts[3]
- Apple Mac Keyboard Symbols[4]

[3] https://fonts.google.com/specimen/Fira+Code?preview.text=%E2%8C%98%E2%87%A7%E2%8C%A5%E2%8C%83%E2%86%A9&query=fira+code

[4] https://support.apple.com/en-hk/guide/mac-help/cpmh0011/mac

```
#let mac-key = keyle.mac-key
#let kbd = keyle.config(theme: keyle.themes.standard)
#let mac-key-font = (
  "FiraCode Nerd Font Mono"
)
#grid(
  columns: (2fr, 1fr, 2fr, 1fr),
  rows: 2em,
  align: horizon,
  ..mac-key.pairs().map(item => (
    raw("#mac-key." + item.at(0)),
    kbd(
      text(font: mac-key-font, item.at(1))
    ),
  )).flatten()
)
```

`#mac-key.command`	⌘	`#mac-key.shift`	⇧
`#mac-key.option`	⌥	`#mac-key.control`	⌃
`#mac-key.return`	↩	`#mac-key.delete`	⌫
`#mac-key.forward-delete`	⌦	`#mac-key.escape`	⎋
`#mac-key.left`	←	`#mac-key.right`	→
`#mac-key.up`	↑	`#mac-key.down`	↓
`#mac-key.pageup`	⇞	`#mac-key.pagedown`	⇟
`#mac-key.home`	↖	`#mac-key.end`	↘
`#mac-key.tab-right`	⇥	`#mac-key.tab-left`	⇤

IV.2 Linux Biolinum Keyboard Symbols

KEYLE provides symbols for Linux Biolinum Keyboard font.

- Linux Biolinum Keyboard[5]

[5] https://libertine-fonts.org/

```
#let biolinum-key = keyle.biolinum-key
#let kbd = keyle.config(theme: keyle.themes.biolinum)
#grid(
  columns: (5fr, 3fr, 5fr, 3fr),
  rows: 2em,
  align: horizon,
  ..biolinum-key.pairs().map(item => (
    raw("#biolinum-key." + item.at(0)),
    kbd(item.at(1)),
  )).flatten()
)
```

`#biolinum-key.Strg`	Strg	`#biolinum-key.Alt`	Alt
`#biolinum-key.Ctrl`	Ctrl	`#biolinum-key.Shift`	⇧
`#biolinum-key.Tab`	⇤⇥	`#biolinum-key.Enter`	↵
`#biolinum-key.Capslock`	⇩	`#biolinum-key.Home`	Home
`#biolinum-key.Del`	Del	`#biolinum-key.Ins`	Ins
`#biolinum-key.End`	End	`#biolinum-key.Space`	
`#biolinum-key.Esc`	Esc	`#biolinum-key.PageUp`	Page↑
`#biolinum-key.PageDown`	Page↓	`#biolinum-key.Back`	←
`#biolinum-key.Pad_0`	Pad 0	`#biolinum-key.Pad_1`	Pad 1
`#biolinum-key.Pad_2`	Pad 2	`#biolinum-key.Pad_3`	Pad 3
`#biolinum-key.Pad_4`	Pad 4	`#biolinum-key.Pad_5`	Pad 5
`#biolinum-key.Pad_6`	Pad 6	`#biolinum-key.Pad_7`	Pad 7
`#biolinum-key.Pad_8`	Pad 8	`#biolinum-key.Pad_9`	Pad 9
`#biolinum-key.Pad_Div`	Pad ÷	`#biolinum-key.Pad_Add`	Pad +
`#biolinum-key.Pad_Sub`	Pad −	`#biolinum-key.Pad_Mul`	Pad ×
`#biolinum-key.Pad_Enter`	Pad↵	`#biolinum-key.Mac_Cmd`	⌘
`#biolinum-key.Mac_Opt`	⌥	`#biolinum-key.Mac_FDel`	⌦
`#biolinum-key.Mac_Del`	⌫	`#biolinum-key.GNU`	
`#biolinum-key.Win`		`#biolinum-key.Tux`	
`#biolinum-key.delim_plus`	+	`#biolinum-key.delim_minus`	-

IV.3 SVG Key Glyphs

A key symbol is just content, so non-textual keys can be passed as inline SVG glyphs from `#keyle.svg-key`. Available names are `up`, `down`, `left`, `right`, `enter`, `backspace`, `tab` and `win`.

```
#let kbd = keyle.config(theme: keyle.themes.flowbite)
#kbd(keyle.svg-key.up) #kbd(keyle.svg-key.down)
#kbd(keyle.svg-key.left) #kbd(keyle.svg-key.right)
#kbd(keyle.svg-key.enter) #kbd(keyle.svg-key.backspace)
#kbd(keyle.svg-key.tab) #kbd(keyle.svg-key.win)
```

Part V

Available commands

#config(⟨theme⟩: themes.standard, ⟨compact⟩: false, ⟨delim⟩: "+") ⟶ function

Config function to generate keyboard rendering helper function.

Argument

⟨theme⟩: themes.standard — function

The theme function to use. Any preset from `themes`, optionally extended with `.with(...)`, or a custom `sym => content` function.

Argument

⟨compact⟩: false — bool

Whether to render keys in a compact format.

Argument

⟨delim⟩: "+" — str

The delimiter to use between keys.

#gen-examples(⟨kbd⟩) ⟶ content

Generate examples for the given keyboard rendering function.

Argument

⟨kbd⟩ — function

The keyboard rendering function.

#join-keys(⟨keys⟩, ⟨theme⟩, ⟨delim⟩) ⟶ content

Join rendered keys with a delimiter between them.

#_len(⟨v⟩) ⟶ length

Coerce a number (treated as points) or a length into a length.

```
#_rect(
  (x),
  (y),
  (w),
  (h),
  (r),
  (fill),
  (stroke): none,
  (sw): 0
) ⟶ bytes
```

Build a parameterized keycap SVG (pure vector, no `<foreignObject>`). All sizes are in points; colors are CSS hex strings or `none`.

```
#keycap(
  (fill): rgb("#eeeeee"),
  (stroke): rgb("#555555") + 0.6pt,
  (radius): 2pt,
  (inset): (x: 3pt, y: 2.5pt),
  (raise): 1.2pt,
  (shadow): rgb("#555555"),
  (baseline): 0.1em,
  (text-args): (fill: black),
  (wrap): it => it,
  (sym)
) ⟶ content
```

Keycap factory using vector rectangles (the rect backend).

Bind styling with `.with(...)` to obtain a theme renderer `sym => content`.

Argument

`(fill): rgb("#eeeeee")` — color

Face fill color.

Argument

`(stroke): rgb("#555555") + 0.6pt` — stroke

Face border.

Argument

`(radius): 2pt` — length | ratio

Corner radius.

Argument

`(inset): (x: 3pt, y: 2.5pt)` — length | dictionary

Inner padding around the symbol.

Argument

`(raise): 1.2pt` — `length`

Diagonal drop-shadow extent; `0pt` renders a flat cap.

Argument

`(shadow): rgb("#555555")` — `color` | `none`

Shadow color, or `none` to disable.

Argument

`(baseline): 0.1em` — `length` | `ratio`

Baseline shift of the cap relative to surrounding text.

Argument

`(text-args): (fill: black)` — `dictionary`

Text layer arguments spread into `text`.

Argument

`(wrap): it => it` — `function`

Text layer content transform.

Argument

`(sym)` — `any`

The key symbol.

`#style-text((sym), (args): (:), (wrap): it => it)` → `content`

Text layer: turn a key symbol into styled content.

`wrap` may be any `content -> content` function, so builtin transforms can be passed directly, e.g. `wrap: smallcaps` or `wrap: upper`.

Argument

`(sym)` — `any`

The key symbol.

Argument

`(args): (:)` — `dictionary`

Arguments spread into `text`.

Argument

`(wrap): it => it` — `function`

Content transform applied after styling.

```
#svg-keycap(
  (fill): rgb("#f3f4f6"),
  (stroke): rgb("#d1d5db"),
  (stroke-width): 1.2,
  (radius): 6,
  (lip): 3,
  (shadow): auto,
  (pad): (x: 6pt, y: 3.5pt),
  (baseline): 0.3em,
  (text-args): (fill: rgb("#1f2937"), weight: "semibold"),
  (wrap): it => it,
  (sym)
) → content
```

Keycap factory using a generated SVG shell (the svg backend).

The SVG is stretched to the measured symbol size so corners never distort. Bind styling with `.with(...)` to obtain a theme renderer `sym => content`.

Argument

`(fill): rgb("#f3f4f6")` — color

Face fill color.

Argument

`(stroke): rgb("#d1d5db")` — color

Face border color.

Argument

`(stroke-width): 1.2` — float

Border width in points.

Argument

`(radius): 6` — float

Corner radius in points.

Argument

`(lip): 3` — float

Bottom shadow lip height in points.

Argument

`(shadow): auto` — color | auto | none

Shadow color, `auto` to derive from `stroke`, or `none`.

Argument

`(pad): (x: 6pt, y: 3.5pt)` — dictionary

Inner padding around the symbol.

Argument

`(baseline): 0.3em` — `length` | `ratio`

Baseline shift of the cap relative to surrounding text.

Argument

`(text-args): (fill: rgb("#1f2937"), weight: "semibold")` — `dictionary`

Text layer arguments spread into `text`.

Argument

`(wrap): it => it` — `function`

Text layer content transform.

Argument

`(sym)` — `any`

The key symbol.

Part VI

Index

C

G

J

K

S

_